



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Tarea 3 – Simulación de algoritmos de planificación de CPU

EQUIPO

Angélica Yin Len Wong Mestas - 317160741

PROFESORES

Prof. Javier León Cotonieto
Ayud. Manuel Tiburcio

ASIGNATURA

Sistemas Operativos

September 21, 2026

MLFQ

Es un algoritmo de planificación de prioridad dinámica. Su objetivo es optimizar de manera simultánea dos métricas sin conocer de antemano el tiempo de ráfaga de CPU de cada uno de los procesos: el tiempo de retorno para trabajos cortos y el tiempo de respuesta para trabajos interactivos.

La idea fundamental es: utilizar el comportamiento histórico de un proceso para predecir su comportamiento futuro. Si un proceso cede la CPU repetidamente, se interpreta como interactivo y se mantiene en alta prioridad, pero si consume su quantum completo una y otra vez, se interpreta como intensivo en CPU y se degrada su prioridad.

"MLFQ intenta optimizar tanto el tiempo de respuesta como el tiempo de retorno sin conocer de antemano el tiempo de ráfaga de CPU de los procesos."

¿Cómo funciona cada algoritmo?

El algoritmo mantiene múltiples colas, cada una asociada a un nivel de prioridad. Un proceso en un cola de mayor prioridad siempre se ejecuta antes que cualquier proceso en una cola de menor prioridad.

Según lo descrito en Operating System Concepts (10ª edición), el comportamiento de MLFQ se rige por las siguientes reglas:

Regla 1: Si $Prioridad(A) > Prioridad(B)$, A se ejecuta y B espera

Regla 2: Si $Prioridad(A) = Prioridad(B)$, A y B se ejecutan en Round Robin

Regla 3: Todo proceso nuevo entra en la cola de mayor prioridad

Regla 4: (Decisión tomada): Una vez que un proceso agota su cuota de tiempo en un nivel (independientemente de si cedió la CPU voluntariamente), su prioridad se reduce un nivel

Regla 4: (Original): Si el proceso agota su quantum completo, baja de prioridad; si cede la CPU antes de agotarlo, conserva su prioridad

Regla 5: Después de un periodo de tiempo S , todos los procesos se reinsertan en la cola de mayor prioridad

La regla 4 (Original) presenta una vulnerabilidad, ya que un proceso podría iniciar una operación de I/O justo antes de que expire su quantum y así "engañar" al planificador para permanecer indefinidamente en alta prioridad; por ello se propuso la Regla 4 (Decisión tomada) la cual emplea una contabilidad acumulativa donde se registra el tiempo total de CPU consumido en ese nivel, sin

importar si hubo cesiones voluntarias, y así si el proceso alcanza la cuota, entonces el proceso se degrada (pasa a la siguiente cola de prioridad inferior secuencial).

La regla 5, es indispensable ya que con el reinicio periódico se resuelven los dos factores: la inanición (si existen muchos procesos interactivos ocupando permanentemente las colas altas, los procesos largos nunca reciben CPU) y la incapacidad de adaptación (un proceso que era intensivo en CPU y luego cambia a ser interactivo quedaría atrapado en las colas bajas).

El rendimiento de MLFQ depende de varios parámetros ajustables: número de colas, longitud del quantum en cada cola (típicamente los quantums de colas altas son cortos y los de colas bajas son largos), intervalo de promoción S , y el método exacto de contabilidad para la degradación.

Estructuras de datos utilizadas:

```
3
4  #define MAX_PROCESOS 200    // por si quieren simular mas de 100
5
6  typedef struct {
7      int id;                // ID del proceso
8      int arrival;           // Tiempo de llegada (t0, t1... tn)
9      int burst;             // Ráfaga de CPU original (tiempo cpu)
10     int priority;           // Tiempo restante (para RR, SRTN)
11     int remaining;         // Para algoritmos con prioridades
12     int start_time;        // Primer instante en que se ejecuta
13     int finish_time;       // Momento en que termina
14     int waiting_time;      // Tiempo total de espera
15     int turnaround;        // finish_time - arrival
16     int response_time;     // primer instante en que el proceso reci
17 } Process;
18
19 // PARÁMETROS MLFQ
20
21 // ESTRUCTURAS AUXILIARES
22 typedef struct {
23     int items[MAX_PROCESOS];
24     int head;
25     int tail;
26     int count;
27 } MLFQ_Queue;
28
29 // Arreglo paralelo con metadatos de cada proceso para MLFQ
30 typedef struct {
31     int queue_level;        // 2 = alta, 1 = media, 0 = baja
32     int cpu_used_in_level;  // Tiempo acumulado en el nivel actual
33 } MLFQ_Meta;
34
35
```

Los datos del archivo de entrada proporcionado `Procesos.txt` no incluyen información sobre comportamiento de I/O, por lo que esta simulación no reflejara la preferencia de MLFQ por procesos interactivos, solo modelara cargas de trabajo puramente intensivas en CPU.

REPOSITORIO:

https://github.com/yinwm88/SO_Algoritmo-MLFQ-

Decisiones tomadas durante la implementación

1. Se eligieron 3 colas de prioridad (Q2 la más alta, Q0 la más baja).
2. Se adoptó la estrategia de quantum corto en alta prioridad, quantum largo en baja prioridad.
3. Se optó por disparar la Regla 5 cada 20 unidades de tiempo, reinsertando todos los procesos listos en Q2. En la Q2, ya que un valor demasiado corto diluye el efecto del mecanismo de prioridades, y uno demasiado largo no evita eficazmente la inanición.
4. Se implementó la Regla 4 optando por contabilidad acumulativa en lugar de quatum. Es decir, aunque un proceso ceda la CPU antes de agotar su quantum, si su tiempo acumulado en ese nivel alcanza la cuota(contabilidad acumulativa), se degrada.

Resultados

Dado que los datos de entrada solo contienen tiempos de ráfaga de CPU sin características de I/O, entonces todos los procesos en MLFQ terminan eventualmente descendiendo desde Q2 hasta Q0.

En general, ya que los datos de entrada solo contiene tiempos de ráfaga(carga puramente intensiva en CPU), entonces MLFQ se comporta de manera similar a una planificación de prioridad multinivel. Los procesos de ráfaga corta (burst=2, 4) completan en Q2 o Q1, mientras que los de ráfaga larga (burst=8, 10) descienden hasta Q0. No obstante, debido a la promoción de prioridad, los procesos largos en Q0 son reinsertados periódicamente en Q2, lo que introduce cierta sobrecarga de planificación e impredecibilidad.

Ejemplo:

- Proceso 1 (arrival=0, burst=2): llega en el instante 0 y se inserta en Q2 (quantum=2). Como burst=2 coincide exactamente con el quantum de Q2, completa su ejecución sin degradarse. Tiempo de respuesta = 0 - 0 = 0; tiempo de retorno = 2 - 0 = 2.
- ...
- Proceso 6 (id=6, arrival=10, burst=2): cuando llega, Q2 puede contener otros procesos según el estado previo de la planificación. Si Q2 está vacía, se ejecuta de inmediato y completa, con tiempo de respuesta prácticamente nulo. Sin embargo, dado que muchos procesos llegan con intervalos de solo 2 unidades de tiempo, varios deben esperar en cola.

```
===== PROMEDIOS (MLFQ) =====  
Tiempo de espera promedio      : 253.52  
Tiempo de retorno promedio     : 259.52  
Tiempo de respuesta promedio   : 68.00
```

En conclusión, MLFQ es un algoritmo de planificación adaptativo cuya principal fortaleza radica en que no requiere conocer de antemano los requerimientos de CPU de los procesos, sino que ajusta dinámicamente las prioridades observando el comportamiento histórico.

Dado el archivo de de entradas, no fue posible exhibir la preferencia de MLFQ por procesos interactivos, que es precisamente uno de sus objetivos más importantes.

References

- [1] Silberschatz, A., Galvin, P. B., Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley. Capítulo 5: CPU Scheduling — Multilevel Feedback Queue.